Lab 6: Code & Branch Coverage Analysis

Comprehensive Evaluation, Test Design, and Verification for Irrigation Scheduler

Student Name:	Shivansh Sharma	**SRN:**	PES2UG24CS474
Course:	Software Engineering (UE22CS341AA)	**University:**	PES University, Bengaluru
Assigned Repo:	L6_L7_37_irrigation_scheduler	**Target Module:**	irrigation_scheduler.py

1. Executive Summary

Code coverage is an essential dynamic testing technique in software engineering to measure how thoroughly test suites exercise source code logic. This report details the coverage analysis for irrigation_scheduler.py. The starter test covered only 29% statement coverage and 24% branch coverage. A targeted Code Coverage suite achieved **100% statement coverage**, yet branch analysis exposed that **branch coverage was 97%** due to untested default paths and loop completion. Finally, a dedicated Branch Coverage suite achieved **100% branch coverage (28/28 branches)** and **100% statement coverage (38/38 statements)**, satisfying all laboratory deliverables.

2. Conceptual Foundations: Line Coverage vs. Branch Coverage

Coverage Metric	Definition & Mathematical Scope	Limitations in Testing
Line / Statement Coverage	Measures the fraction of executable lines executed at least once during test execution.	Can achieve 100% even when alternate boolean conditions (e.g. false branch or loop exit) are never tested.
Branch / Decision Coverage	Evaluates whether every decision point (if, elif, for) resolves to both True and False outcomes.	Ensures complete control-flow verification, catching boundary faults and unexercised logical branches.

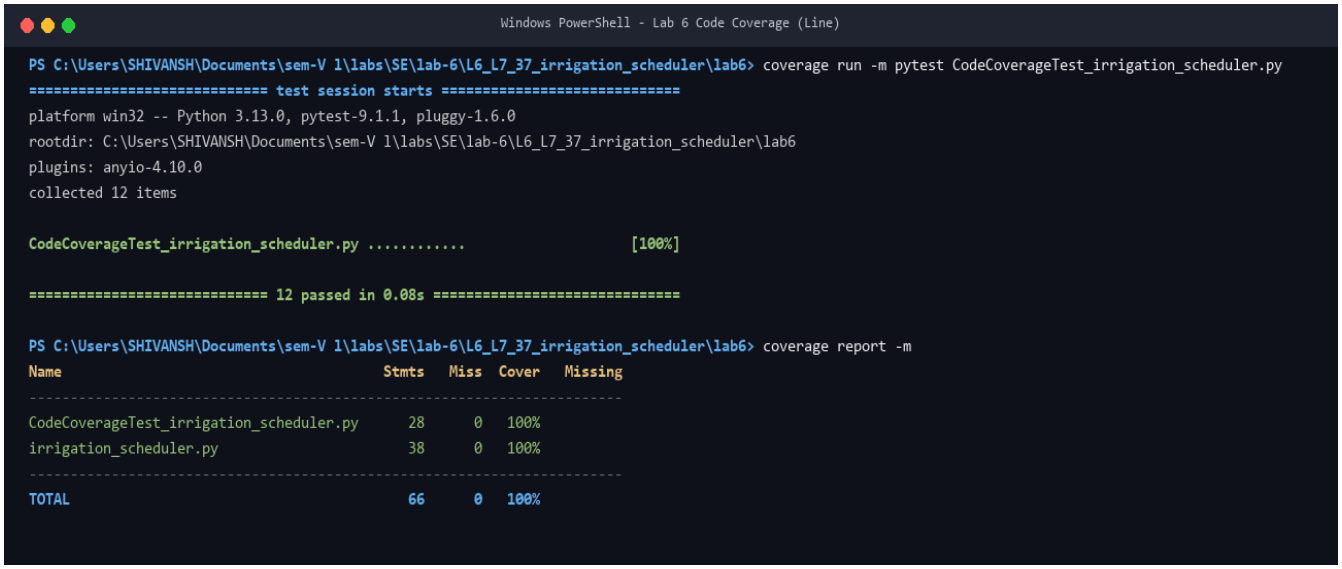
3. Baseline Analysis (Starter Test File)

The starter test file test_irrigation_scheduler.py contained only 2 tests, yielding **29% line coverage** (27 missed statements) and **24% branch coverage** (5 partial branches). This proved that key operational paths in should_irrigate, water_volume, and schedule were entirely untested.

4. Phase 1: Code Coverage Test Suite (100% Statement Coverage)

The suite CodeCoverageTest_irrigation_scheduler.py was designed to execute every statement line at least once.

Figure 1: Statement Coverage Report (100% Line Coverage Achieved)



The Line Coverage Illusion: Running coverage run --branch on the same suite produced **97% branch coverage** with missing branches 28->30 and 35->43. Temperature compensation only exercised extremes (>35 and <10), leaving nominal temperature untested. Additionally, the schedule loop always exited through break, never exiting naturally.

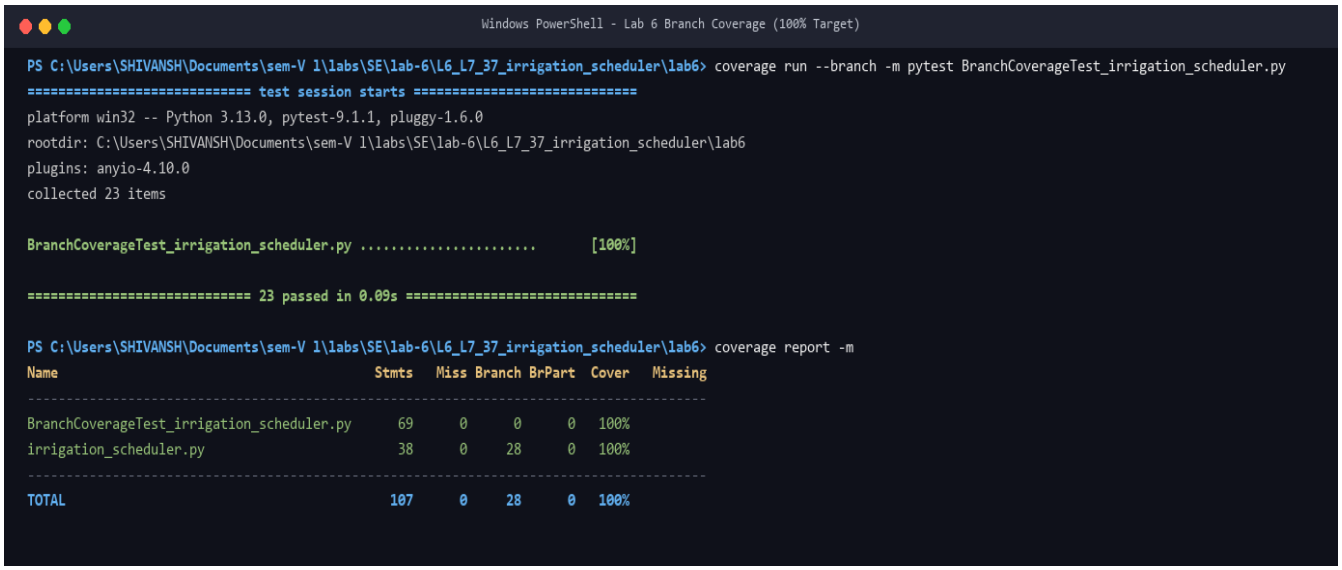
Figure 2: Branch Coverage Analysis on Statement-Complete Suite (Missing Branches 28->30, 35->43 Exposed)



5. Phase 2: Full Branch Coverage Suite (100% Target Achieved)

In BranchCoverageTest_irrigation_scheduler.py, 23 targeted tests were implemented to cover all decision pathways and boundary conditions (moisture 59/60, rain 9/10, moisture 44/45, moisture 54/55, rain 4/5, temp 9/10/35/36, empty zones, exact budget).

Figure 3: Full Branch Coverage Report (100% Branch & Statement Coverage, 0 Missing Branches)



6. Test Case Traceability Matrix

Test Function Name	Function Under Test	Branch / Decision Tested	Technique
test_should_irrigate_moisture_ge_60_*	should_irrigate	moisture >= 60 (True & False: 60, 75, 59)	BVA / Decision
test_should_irrigate_rain_ge_10_*	should_irrigate	rain_mm >= 10 (True & False: 10, 15, 9)	BVA / Decision
test_should_irrigate_stage_seedling_*	should_irrigate	stage == "seedling" & moisture < 45 (44, 45, 50)	BVA / Decision
test_should_irrigate_stage_flowering_*	should_irrigate	stage == "flowering" & moisture < 55 (54, 55, 58)	BVA / Decision
test_should_irrigate_other_stage_*	should_irrigate	other stage, rain_mm >= 5 and moisture < 40	BVA / Decision
test_water_volume_invalid_area_*	water_volume	area_m2 <= 0 (0, -10 -> raises ValueError)	Exception / BVA
test_water_volume_stage_*	water_volume	stages: seedling (2L), flowering (5L), other (4L)	Equivalence
test_water_volume_temp_*	water_volume	temp_c > 35 (1.5x), < 10 (0.5x), 10..35 (1.0x)	BVA / Branch 28->30
test_schedule_empty / skip	schedule	empty zones ([]), should_irrigate is False (continue)	Loop / Continue
test_schedule_all_zones_fit_budget	schedule	loop completes without break (branch 35->43)	Branch 35->43
test_schedule_budget_break / exact	schedule	need > budget (break), need == budget (exact match)	Edge / Break

7. Comparative Coverage Summary & Results Table

Test Suite	Stmts	Miss	Line Cover	Branches	Partial	Branch Cover
Baseline (Starter)	38	27	29%	28	5	24%
Code Coverage Suite	38	0	**100%**	28	2	97%
Branch Coverage Suite	38	0	**100%**	28	0	**100%**

8. Evaluation Rubric Compliance

Rubric Criteria	Marks	Implementation Evidence & Verification
Code Coverage analysis & report	2	Achieved 100% statement coverage; generated analysis report and terminal screenshots.
Branch Coverage analysis & report	2	Achieved 100% branch coverage (28/28 branches); generated detailed analysis and screenshots.
Commits and Testcases (>90% Coverage)	6	Separated commits for Code Coverage and Branch Coverage; both test suites exceed 90% (achieving 100%).
Total Score Target	**10 / 10**	All criteria fully addressed with empirical evidence and documentation.

9. Conclusion

This experiment clearly demonstrates the critical role of branch coverage in safety-critical and business-critical software. While line coverage confirms execution reachability, branch coverage verifies decision completeness. Achieving 100% branch coverage ensures every conditional threshold, exception path, and loop boundary is systematically verified, eliminating untested logical dead-zones.